实验指导：RSA 共享素因子攻击

0. 实验概述

0.1 实验内容

有 1000 台设备的 RSA 公钥摆在面前。**你不需要分解任何一个 1024 位的数** —— 只要做一次 499,500 对的最大公约数扫描，就能把其中若干台设备的私钥**完整恢复出来**，并用它解开一份加密的 PDF。

背景事件：2012 年，两组研究者对全网做了一次扫描，一次就拆出了数万台设备的私钥。你这次要做的，是同一件事的**缩小版**，规模从十万降到一千 —— 但用的数学分解方式没有变化。

0.2 部分说明

> 本文的「实测输出」块里，凡是属于答案的**设备编号**和**那几个计数**，都写成 `[···]`。这不是排版偷懒 —— 那些数字要**你自己跑一遍**才知道。

---

1. 原理速览

1.1 一句话原理

RSA 的安全性建立在「大整数分解很难」之上：

```
n = p · q          p、q 都是 512 位素数
公开 (n, e)        私钥 d = e⁻¹ mod φ(n)，其中 φ(n) = (p−1)(q−1)
```

攻击者的目标是拿到 p、q。分解一个 1024 位的合数，用目前最好的算法和机器，代价远超本实验的课时 —— **这条路走不通，也不该走。**

但这个实验里的 1000 台设备**不是彼此独立的。**它们由各自的随机数发生器 生成，而随机数发生器会出错。**一旦有两台设备用了同一个素数**，攻击者 就完全不需要分解了：

```
n₁ = p · q₁
n₂ = p · q₂
gcd(n₁, n₂) = p          ← 最大公约数直接把 p 交出来
```

拿到 p 之后，$q_1 = n_1 / p$，$\varphi(n_1) = (p-1)(q_1-1)$，$d_1 = e^{-1} \bmod \varphi(n_1)$。 **私钥到手，全程没有任何「分解」发生。**

这就是共享素因子攻击（shared prime attack），也叫批量 GCD 攻击。

1.2 它为什么真实存在

关键在于：**RSA 的数学没有坏，坏的是随机数。**

素数不是「想出来」的，是**随机生成**的：抽一个 512 位的随机数，测它是不是素数，不是就再抽。如果两台设备的随机数发生器在某个时刻给出了**相同或高度相关的 输出**，它们就会选中同一个素数。

真实的成因有好几种：

成因	后果
嵌入式设备启动时熵池还没攒够，`/dev/urandom` 尚未就绪	两台设备拿到同一段「随机」数
设备出厂时**克隆**了同一份密钥镜像，事后只重烧了一部分	模数不同，但素因子重复
随机数发生器状态被**完整复用**（整机镜像、快照复原）	两台设备的模数**一模一样** —— 异常，但拆不开
实现里 $n = p^2$（同一个素数用了两次）	模数成为一个**完全平方**

前两类会让两台设备**共享一个素数**（本实验主语料的类型）； 第三类会让模数**重复**；第四类会让模数成为**完全平方**。

本实验的把这三类都放了进去 —— 因为**只有当你亲手遇到过每一类， 你才知道要做哪几项检查。**

1.3 这条路线的两个陷阱

(1)「异常」和「可分解」不是一回事。

有些设备确实「不正常」—— 比如两台设备的模数一模一样。这当然是事故， 报告必须写。但攻击者从「两个相同的 n」里**什么也拿不到**： 他知道 n，他本来就知道 n。拿不到 p、q，就拿不到 d。

把「异常」和「可恢复私钥」混成一个数字，会导致一个真实的后果： 运维方拿着一份清单去紧急轮换一批**根本不危险**的密钥，而真正危险的那批 淹没在里面。

(2) 一个工具只能看见它问的那个问题。

两两 GCD 对 $n = p^2$ **完全沉默** —— 它的两个因子是同一个 p， 没有别的设备持有 p，所以 `gcd(它，谁) == 1`。

扫描跑完，一条命中都没有。但它是**可以分解的。**

1.4 要用到哪些数学

数学工具	用在哪
最大公约数 gcd	核心武器（`math.gcd`，C 实现）
扩展欧几里得 egcd	求模逆（$d = e^{-1} \bmod \phi$）
模逆 modinv	把公开指数 e 变成私钥 d
欧拉函数 φ	φ(pq) = (p−1)(q−1)；**$\varphi(p^2) = p(p-1)$**
欧拉定理	解释 $m^{ed} \equiv m \pmod n$ 为什么成立
整数平方根 isqrt	找完全平方模数（两两 gcd 看不见的那一类）
PKCS#1 v1.5 填充	任务四把会话密钥取出来的那一步

---

2. 环境与数据

2.1 环境要求

不需要安装任何东西。 只需要 Python 3.10 或更高版本。

```
python --version        # 应当 >= 3.10
```

本包的 code/ 目录**只用 Python 标准库**，连 AES 都是随包附带的纯 Python 实现（code/aes_soft.py）。这一条是刻意的：机房通常不让装东西， 而实验课最不该浪费的时间就是在装包上。

先跑环境自检（这是课前作业）：

```
cd rsa-lab4/code
python check_env.py
```

它会检查 9 项：解释器版本、运行方式、终端编码、数据文件、语料完整性、 AES 的 NIST 官方向量、PBP 格式往返、RSA 公共工具、填空机制。 全绿之后，它还会印出**填空进度表**（29 处空），收尾句是「全部通过 —— 可以开始实验了。」 这 90 分钟里的头 6 分钟，**只处理自检没过的机器** —— 自检没跑的， 请课前补上，别占用课堂时间。

> ⚠ **绝对不要用 `python -I` 或 `python -E` 运行本包的脚本。** `-I` 会把脚本所在目录移出 `sys.path`，`from fill import ...` 会直接失败。 `check_env.py` 的第 2 项专门检查这个，被拦住了按提示改回来即可。

2.2 目录结构

```
rsa-lab4/
├── code/                    ← 你要改的代码都在这里
│   ├── fill.py              ★ 填空机制。★ 本文件不要修改 ★
│   ├── check_env.py         环境自检（课前作业）
│   ├── rsa_common.py        共用工具：承诺值 / I2OSP / PKCS#1 解码
│   ├── pbp_common.py        PBP 文件格式（只读黑盒，不要改）
│   ├── aes_soft.py          纯标准库 AES（零依赖的关键）
│   ├── decrypt_hw.py        黑盒解密容器（--auto 是兜底）
│   ├── task1_math.py        【填空 10】扩展欧几里得 + 模逆 + φ
│   ├── task2_pair.py        【填空 6】两两扫描 + 拆开一对
│   ├── task3_scan.py        【填空 9】分类 + 完全平方 + 恢复私钥【主交付物】
│   ├── task4_decrypt.py     【填空 4】解开 PBP，拿回 PDF
│   ├── verify.py            自查工具（性质检查，不告诉你答案）
│   └── run_all.py           一键跑通 + 填空进度表
└── data/
    ├── moduli.txt               1000 个模数，每行一个十六进制数
    └── homework.pdf.enc.asc     加密的 PBP 容器（任务四的目标）
```

2.3 数据说明

文件	大小	说明
moduli.txt	257,000 字节	1000 行十六进制。**位长有 1023 和 1024 两种**，这是正常的，不要为它写「每行必须 256 个字符」这类校验
homework.pdf.enc.asc	23,136 字节	一个 PBP 文件（Pretty Bad Privacy，本包自带的教学容器）：一个二进制 blob 的 base64/ASCII 封装，有 BEGIN/END 头尾标记

moduli.txt 的第一行长这样（**只是格式示例，不是语料里的真实值**）：

```
0x9f3c...（共约 256 个十六进制字符）
```

3. 场景

你被请去做一次**密钥健康检查**。

某单位有一批自研的加密设备，出厂时各自生成 RSA 密钥对，公钥登记在册。 最近上级要做一次安全摸底，你拿到的就是这**1000 台设备的公开模数清单** （`data/moduli.txt`）。

同时，运维方交给你一份**加密的通知文件**（`data/homework.pdf.enc.asc`）。 它用的是 PBP 格式（Pretty Bad Privacy —— 本包自带的一种密钥容器： 把一份文件**加密给某把 RSA 公钥**）。里面的会话密钥被 **其中一台设备的 RSA 公钥**加密了。运维方说：这台的私钥已经随设备下线丢失了。

你要做两件事：

1. **找出哪些设备不安全**，把它们的私钥恢复出来，交出一份结构化的结果；
2. **用恢复出的私钥解开那份通知。**

第一件事不是「写个脚本跑一次」—— 你得把「命中的**对**」翻译成「有问题的**设备**」， 把两两 GCD **看不见**的那一类补上，还要分清「异常」与「可恢复私钥」。

4. 填空怎么填

4.1 填空长什么样

本实验的代码骨架是**完整可运行**的，只在关键位置留了空。一个空长这样：

```
g = FILL[2]                          # ← 表达式位：把 FILL[2] 换成你的表达式
```

或者这样：

```
# TODO(4): 把这一对记下来 —— 设备号 i、设备号 j，以及 g。
FILL[4]                              # ← 语句位：把 FILL[4] 删掉，原位写你的语句
```

每个 TODO 都标了编号，编号与本文 §5 和附录 E 一一对应。

`FILL` 是 `fill.py` 里 `Blanks` 类的一个实例 —— 只要你运行到那一行， 它就会抛出一个专门的异常：

```
FILL[2] not filled yet -- task2_pair.py 第 96 行:
    | g = FILL[2]
把 FILL[2] 这几个字删掉，在原位写你的代码。
```

报错信息直接告诉你文件名和行号，不需要你自己去找。

4.2 三条注意事项

一：把 `FILL[n]` 这几个字符删掉，在原位写代码。不要给它赋值。

```
FILL[4] = math.isqrt(n)      # ✗ 错！会抛 TypeError
r = math.isqrt(n)            # ✓ 对！把 FILL[4] 删掉换成这个
```

FILL[4] = ... 会被告知：「不要给 FILL[4] 赋值 —— 把 FILL[4] 这几个字删掉， 在原位直接写你的代码。」这是因为赋值只是把那个名字绑到了别的东西上， 无法判断你填得对不对。

二：一处填空 = 一个完整的表达式或一条完整的语句。

```
hits.append(FILL[4])             # ✓ 对
r = FILL[4]                      # ✓ 对
FILL[2]                          # ✓ 对（语句位，整行替换）
```

不要嵌进更大的表达式里 —— 比如 2 * FILL[1] + 1 这种写法在本包中不会出现， 因为文本替换有优先级陷阱。如果你看到需要这样写，说明骨架写错了， 告诉老师。

三：填空只出现在函数体里，不在模块顶层。

所以任何文件都**可以随时 import、可以 --help**，只有真正执行到那一行才会失败。 这也是为什么一个文件里没填完不影响另一个文件被 import。

4.3 怎么知道还剩几个空

```
python run_all.py --progress
```

输出（**未填时**）：

```
================================================================
rsa-lab4 填空进度
================================================================

  task1_math.py       ▒▒▒▒▒▒▒▒▒▒  0 / 10
  task2_pair.py       ▒▒▒▒▒▒  0 / 6
  task3_scan.py       ▒▒▒▒▒▒▒▒▒  0 / 9
  task4_decrypt.py    ▒▒▒▒  0 / 4

  合计 0 / 29
  没填完不要紧：跑到那一行会告诉你具体是哪个文件的第几行。
  （就像这样：FILL[2] not filled yet -- task2_pair.py 第 96 行）
```

直接跑 python run_all.py 则是一键顺序跑完所有步骤。**没填完时它会停在 第一处没填的空上，并打印进度表 —— 这是预期行为，不是坏了。**

用 grep -c 'FILL\[' 数空**是不准的**：提示注释里也会出现 FILL[2] 这样的字样， 一处空可能被数成三处。以 run_all.py --progress 的输出为准（它用的是语法树）。

4.4 「还没填」和「填错了」是两回事

本包对这两种情况给了**完全不同的表现**：

情况	退出码	表现

空还没填	**3**	`FILL[n] not filled yet` + 文件名 + 行号 + 怎么修
填错了	**1**	自检里的具体失败项 + 针对性提示
全对	0	全部通过

退出码 3 的异常类叫 `BlankNotFilled`，它**故意继承 `BaseException`** —— 这样任何 `except Exception` 都吞不掉它。你不必担心自己写的 `try/except` 会把「还没填」变成一个看不懂的错误。

---

5. 四个任务

每个任务固定五段：**目的 → 命令 → 实测输出 → 要求 → 关键结论。**

5.1 任务一　数学地基：扩展欧几里得、模逆与 φ【10 处填空】

目的

整条攻击链的终点是「把公开指数 e 变成私钥 d」，而那个变换就是模逆：

```
d = e⁻¹ mod ϕ(n)
```

这个式子里有**两样**东西都得你自己写：**模逆**（`modinv`）和**分母 φ(n)**（`totient`）。如果它们是别人给你的，那这次攻击最关键的一步你就说不清了 —— 你只是按了一个按钮。所以这两个函数，加上它们脚下的 `egcd`，三个函数一共约 10 行，全归你写。

命令

```
python task1_math.py --selftest
```

实测输出（本机 Python 3.14.5 / Windows 10）

```
================================================================
任务一自检：扩展欧几里得 + 模逆
================================================================
【一】Bézout 恒等式  a·x + b·y = gcd(a, b)
  全部 7 例通过

【二】模逆  a · modinv(a, m) ≡ 1 (mod m)
  全部 5 例通过

【三】⚠ 返回值必须在 [0, m) 区间 —— 负 d 的唯一防线
  modinv(3, 7) = 5   通过（落在 [0, 7) 内）

【四】⚠ 逆元不存在时必须抛 ValueError，而不是返回一个错值
  全部 4 例都正确抛出了 ValueError

【五】⚠ φ(n) 的两种情形 —— 错了要到任务四才发作
  全部 3 例通过（含 (p−1)² = 36 ≠ 42 的反例）

----------------------------------------------------------------
自检通过后，顺手想清楚这三问：
  (a) 为什么 gcd(a, m) ≠ 1 时逆元不存在？
  (b) Bézout 恒等式成立，为什么还不足以说明 d 是「合格」的逆元？
  (c) φ(p²) 算错成 (p−1)² 时，程序一声不吭。为什么「d 落在
      [0, φ) 内」这条检查抓不住它？

  (c) 就是报告里任务一那一问，现在就可以先想；
  (a)(b) 想清楚了写在任务一的笔记里。

全部通过 —— 任务一完成，可以带着它上任务二了。
```

（最后一块【五】考的是 φ —— 标题里没写它，但它也在任务一。见下面 (4)。）

要求

空	在哪	写什么
FILL[1]	egcd 循环**之前**	给 (old_s, s) 一个正确的初值
FILL[2]	egcd 循环**之前**	给 (old_t, t) 一个正确的初值
FILL[3]	egcd 循环体	同时更新 (old_s, s)
FILL[4]	egcd 循环体	同时更新 (old_t, t)
FILL[5]	modinv 开头	一次取出 egcd 的三个返回值
FILL[6]	modinv 中段	g != 1 时抛 ValueError 的条件
FILL[7]	modinv 结尾	把 x 归约到 [0, m) 再返回
FILL[8]	totient 开头	判断走哪个分支的条件（拦下的是**普通**情形）
FILL[9]	totient 普通分支	p ≠ q 时的公式
FILL[10]	totient 结尾	p = q（完全平方）时的公式

关键结论（四个，都会考）

(1) 元组赋值不能拆成两句。

```
old_s, s = s, old_s - q * s          # ✓ 对
old_s = s                            # ✗ 错!
s = old_s - q * s                    #     第二句里的 old_s 已被覆盖
```

程序不报错，只是答案全错。 自检里的「Bézout 恒等式」那一组会抓到它 —— 如果 g 和 gcd 对得上、但 `a·x + b·y` 不对，几乎肯定是这里。 `FILL[1]`、`FILL[2]` 也有个同族的坑：初值必须写在循环**之前**， 写进循环体里每轮会被重置，循环跑完永远拿到 (1, 0)。

(2) `modinv` 必须把结果归约到 `[0, m)`。

`egcd(3, 7)` 返回 `(1, -2, 1)`，`x = -2` 是完全合法的 Bézout 系数， 但它不是「逆元」的规范形式。归约一下：`-2 mod 7 = 5`，验算 `3×5 = 15 ≡ 1 (mod 7)` ✓。

这一步为什么不能省：Bézout 恒等式对 `x = -2` 照样成立； 更麻烦的是在 Python 里 `e * d % phi == 1` 对 `d = -2` 也**照样为真** （因为 Python 的 `%` 总返回非负数）。于是你会揣着一个负的 d 一路走到任务四， 直到解密失败才发现问题，而且完全不知道问题出在几百行之前。

自检里的【三】是唯一能抓住它的检查 —— 那一条不是可有可无的。

(3) `gcd(a, m) ≠ 1` 时逆元根本不存在，必须抛 `ValueError`。

在 RSA 里这条就是「e 必须与 φ(n) 互素」。不检查的话你会拿到一个看起来 完全正常的数，而它根本不是逆元。

(4) φ 以前是随包附带的，现在归你写 —— 这是全包后果最重的一处。

在姊妹包 rsa-lab3 里，`totient` 是随包给的，构造上保证正确。现在它搬进了 这个文件：你可能会写错一次，然后在【五】那个检查上花五秒钟改对 —— 而不是在 60 分钟之后、任务四里，对着一份解不开的 PDF 发呆。

陷阱在 `p == q`（完全平方模数 $n = p^2$）那一支：

```
φ(p²) = p² − p = p(p − 1)        ★ 不是 (p − 1)²
```

绝大多数人会顺手只写一条 `(p - 1) * (q - 1)` —— 它在 $p \neq q$ 时完全正确， 在 $p = q$ 时比真值**少了 p − 1**，也就是一个 512 位的数。算出来的 d 不会越界、不会抛异常、`e·d ≡ 1` 也照样成立（模的是错的那个 φ）。 唯一的症状是 60 分钟之后任务四解不开。

⚠ **「d 落在 [0, φ) 内」这条检查抓不住它。** 因为 $\varphi_wrong = (p-1)^2 < p(p-1) = \varphi_true$ —— 错值算出来的 d 本来就更小， 照样落在范围里。

> 顺带一问：`FILL[8]` 的分支条件写反了会怎样？`p != q` 和 `p == q` 两种写法 都有人用，但**必须和两条 `return` 配套**。写反了【五】会当场报 （比如那颗 61、53 的普通用例会算出 3660 而不是 3120）—— 所以这一处**错了会响**，不用担心它悄悄溜过去。

5.2 任务二　拆开一台设备【6 处填空】

目的

把「两两 GCD」真的跑起来，并**亲手拆开其中一对**： 从两个公开模数出发，算出其中一台设备的**完整私钥**，再用它做一次 公钥加密 → 私钥解密的**往返验证**。

往返验证不是走过场 —— 它是「这把私钥是对的」唯一的独立证据。

命令

```
python task2_pair.py
```

（本机约 6~7 秒。第一次跑会稍微慢一点，因为要读 257 KB 的语料。）

实测输出

```
================================================================
任务二：扫描 1000 个模数
================================================================
两两组合数：C(1000, 2) = 499,500
（本机约 6~7 秒，请稍等）

命中         设备 i    设备 j   gcd 的位长
----------------------------------------------------------------
          [···]    [···]   512 位
          [···]    [···]   512 位
          [···]    [···]   512 位
          [···]    [···]   1024 位
          [···]    [···]   512 位

命中条数：[···]
扫描用时：6.44 秒
已写出：...\data\hits.json（任务三会读它）

================================================================
拆开其中一对
================================================================
选中设备 [···] 与设备 [···]
  n_[···] 位长          : 1023
  gcd(n_[···], n_[···])  : 512 位  ← 这就是共享的那个素数
  位长正好是模数的一半，符合「两个 512 位素数之积」的预期

  p                : 512 位
  q                : 512 位
  p · q == n       : 是
  ϕ(n)             : 1023 位
  d                : 1021 位
  d 的承诺值        : 03371cecdb6387e1

================================================================
往返验证：公钥加密 → 私钥解密
================================================================
  m = 7                    -> c = [···] -> m' = 7                    ✓
  m = 12345                -> c = [···] -> m' = 12345                ✓
  m = 18446744073709551617 -> c = [···] -> m' = 18446744073709551617 ✓
  m = 340282366920938463463374607431768211455 -> c = [···] -> m' = 340282366920

往返全部成功 —— 这把私钥是正确的。
注意你刚刚做到的事：没有分解任何 1024 位的数，
只用了两次 gcd（一次在扫描里、一次在 break_pair 里）。
```

> 📌 它选中的是扫描里**第一条** `1 < g < n` 的命中，所以每次跑选中的 都是同一对。上面这 4 个 `m` 是刻意挑的：1 位数、5 位数、$2^{64}+1$、$2^{128}-1$ —— 都小于这台的 `n`，覆盖了「远小于 n」到「接近 n」的情形。

要求

空	在哪	写什么
`FILL[1]`	`scan_pairs` 双重循环内	内层循环的**起点**（无序对只算一次）
`FILL[2]`	`scan_pairs` 双重循环内	求两个模数的最大公约数（一行）
`FILL[3]`	命中判断	什么样的 g 值得记下来（条件表达式）
`FILL[4]`	命中分支内	把 `(i, j, g)` 记进 `hits`（一行）
`FILL[5]`	`break_pair`	把三种 g 的情形分开的**整块代码**
`FILL[6]`	`break_pair` 结尾	`d = modinv(e, phi)`（注意参数的顺序和身份）

`FILL[1]` 的提示：内层从 `i + 1` 开始，否则每个无序对要算两遍， 还会出现 `(i, i)` 这种自己跟自己求 gcd 的项 —— `gcd(n, n) = n`， 会被误读成「重复模数」。

`break_pair` 要处理的三种情形：

情形	含义	怎么办
`g == 1`	两台设备无关	不是这一对，抛 `ValueError` 说清楚
`1 < g < n`	**共享素因子**	这就是我们要的：`p, q = g, n // g`
`g == n`	两个模数**完全相同**	异常，但**拆不开** —— 同样抛 `ValueError`

关键结论

(1) 用 `math.gcd`，不要手写欧几里得。

同一批 300 个模数（44,850 对），本机实测：

实现	总耗时	每对
手写 Python 欧几里得	6.27 ~ 6.77 s	139.7 ~ 150.9 µs
`math.gcd`（C 实现）	0.57 ~ 0.64 s	12.7 ~ 14.4 µs

差一个数量级（连测四轮：9.7 ~ 11.7 倍）。 把这个倍数带到全部 1000 台设备（499,500 对）上：手写版要一分多钟，`math.gcd` 只要 6 秒出头。 同一套数学步骤，只是常数不同 —— 在 90 分钟的一节课里， 常数因子决定你能不能下课前做完。

> 📌 手写版的波动比 `math.gcd` 大得多，这是**可以预期的**： 它是纯 Python 循环跑在 1023/1024 位大整数上，每一轮的除法次数不一样， 而且更容易被机器负载影响。C 实现那一边反而稳。
>
> 这也是「算法复杂度」和「实现常数」是两个概念的第一个例子 —— 同一套数学步骤，常数因子差了一个数量级。

(2) 命中判断只写 `g > 1`，别在这里就把 `g == n` 滤掉。

现在**先把所有异常都抓出来**，怎么分类是任务三的事。在这里就把 `g == n` 过滤掉（写成 `1 < g < n` 之类），你会永远不知道语料里还有重复模数 —— 而「你漏报了一类异常」和「语料里没有这一

类」，看起来是一样的：

- 命中条数会少一条；
- 「命中涉及设备」少两台（重复的那一对整个消失）；
- 任务三的 `duplicate_devices` 变成空集。

最麻烦的是：**`verify.py` 的性质检查抓不住它** —— 往返、承诺值、分类自洽 全都通过，只有加 `--appendix-b` 时「模数重复设备集合」那一项会报对不上。 所以这一处只能靠**你自己知道还有这一类**。报告 §三 有一问就落在这里。

(3) `g == n` 那一行是真的。

上面那张表里有 4 行是 512 位（模数的一半，符合「两个 512 位素数之积」的预期）， 有 1 行是 **1024 位** —— 和模数一样长。这说明 `gcd` 把**整个模数**吐了出来， 也就是两个模数**完全相同**。

这是**异常**，必须写进结论。但攻击者从里面**什么也拿不到**： 他知道 `n`，他本来就知道 `n`。

记住这一行 —— 任务三「三个数别混」的第一个坑就在这里。

(4) 往返验证检验的是欧拉定理。

拿到 `d` 之后，程序拿 4 个不同的 `m` 各做一次「公钥加密 → 私钥解密」。 它成立的理由是 `m^(ed) ≡ m (mod n)` —— 这是 φ(n) 的定义加欧拉定理的直接推论。 **它是「这把私钥是对的」唯一的独立证据**：验证的是性质，不是外观 （「`d` 看起来像个大数」什么也证明不了）。报告 §三 还有一问落在它身上。

5.3 任务三　扫描全部、分类并恢复私钥【9 处填空】【★ 主交付物】

目的

把任务二的一次扫描变成**一次完整的密钥健康检查**：

1. 把「命中的**对**」翻译成「有问题的**设备**」；
2. 补上两两 GCD **看不见**的那一类（`n = p²`）；
3. 恢复**每一台**可分解设备的完整私钥；
4. 写出 `data/weak_devices.json`。

命令

```
python task3_scan.py           # 需要 data/hits.json，由任务二产生
python task3_scan.py --adopt-hits    # hits.json 丢了？用它从语料重建
```

实测输出

```
================================================================
任务三：分析 1000 个模数
================================================================
（hits.json 记录的扫描用时：6.44 秒）
命中 [···] 条

类别                                        台数
----------------------------------------------------------------
g == 1（正常，不含平方）                      [···]
1 < g < n（共享素因子）                       [···]
g == n（模数重复）                            [···]

================================================================
完全平方检测（扫描对它是沉默的）
================================================================
用 isqrt 检查全部 1000 个模数
  设备 [···]：n = 512 位素数的平方   （n 位长 1024）

找到 [···] 台 —— 这一类在扫描里一条命中都没有。

================================================================
★ 三个数，别混
================================================================
  命中条数       [···] 条
  命中涉及设备   [···] 台    [···]
  真正可分解     [···] 台    [···]

  [···] 台里，重复的那 [···] 台拆不开；
  [···] 台里，完全平方那 [···] 台一条命中都没有。
  【命中集合与可分解集合，是两个不同的集合。】

================================================================
恢复私钥
================================================================
    设备   来源            d 的位长    承诺值
----------------------------------------------------------------
   [···]  shared        1021   03371cecdb6387e1
   [···]  shared        1019   9b8bf2362f8a5442
   [···]  square        1024   7c2b2f26bdfe447b
   [···]  shared        1024   1a6a777e738b0d5d
   [···]  shared        1023   6b7ea890b40fb4b0
   [···]  shared        1023   a6283bd6d83be7a6

================================================================
交付物
================================================================
已写出：...\data\weak_devices.json
字节数：4,313

下一步：python verify.py    —— 自查（不告诉你答案，只检查性质）
```

> ⚠ 本文把「三个数」和「承诺值」都写成了 [···] 或占位形式， 展开后的真值在**附录 B。请先自己跑出来，再去看附录 B。**

要求

空	在哪	写什么
FILL[1]	classify	一条命中属于哪一类的条件（★ 全包价值最高的一处空）
FILL[2]	classify（命中分支）	把**设备 i 和设备 j** 都记进 shared
FILL[3]	classify（else 分支）	把设备 i 和设备 j 都记进 duplicate
FILL[4]	find_perfect_squares	n 的整数平方根
FILL[5]	find_perfect_squares	「判断 + 记录」那一整块（两条语句）
FILL[6]	recover_keys（共享素因子分支）	与**搭档**的模数求 gcd，得到共享的素数 p
FILL[7]	同上	n 被 p 整除，商就是另一个素因子
FILL[8]	同上	d = modinv(e, phi)（一行）
FILL[9]	recover_keys（完全平方分支）	phi = totient(p, q)（**p == q 的陷阱**）

FILL[2]/FILL[3] 的容器是 set 不是 list —— 想清楚为什么： 设备会**在多条命中里各出现一次**，用 list 的话台数会被数大。

关键结论

(1) FILL[1] 是全包价值最高的一处空：1 < g < moduli[i]。

两个方向都错得很响：

- 写成 g > 1（或少了上界）→ 重复模数那一类被并进「可分解」： 打印出来的「真正可分解」从 6 台变成 8 台，紧接着在「恢复私钥」那一步 崩掉（重复的模数找不到搭档求 gcd）—— 更要命的是，如果你不理这个， 交给运维方的数字就是错的：**异常 ≠ 可分解**；
- 想清楚：g == 1 不会出现在 hits 里（任务二只收 g > 1）， 所以「不是这一类」就只剩 g == moduli[i] 那一种 —— else 分支只会是重复。

(2)「命中条数」「命中涉及设备」「真正可分解」是三个不同的数。

三个数各自数的是**不同的对象**：

- **命中条数** —— 有几**对**模数被 gcd 判为异常（任务二里 hits 的长度）；
- **命中涉及台数** —— 这些命中合起来牵涉到几**台**设备（把设备号去重后数）；
- **真正可分解** —— 私钥能真正恢复出来的有几台。

条数与台数没有固定的大小关系，因为一台设备可以出现在**多条**命中里：

```
设设备 7、8、9 共用一个素数 p，则 (7,8) (7,9) (8,9) 都会命中：
3 条命中，只涉及 3 台设备；
而 2 台一组的命中只有 1 条，却占 2 台。
```

另外注意：g == n 的重复模数**也在命中表里** —— 它们算命中， 但不算可分解。

第三个数则在另一头与它们分家：

- 完全平方那一台**一条命中都没有**（gcd 对它完全沉默），但它**可以**被分解；
- 模数重复的那几台产生命中，但**拆不开。**

到底是多是少，把你自己输出里的命中表数一遍 —— 数条数，再把两台设备 去重数台数；可分解台数以 `weak_devices.json` 里实际恢复出来的为准。 报告 §四 4.2 的第一问考的就是这个动作。

先把这三个数抄进报告 §四 4.1 的表，再对照附录 B。 报告 §四 还问「三个数为什么不一样」和「完全平方那一台你怎么找出来的」。

(3) `n = p²` 这一类，必须单独查，而且只有 `isqrt` 抓得住。

```
r = isqrt(n)
if r * r == n:      # r 就是它的素因子（两个都等于 r）
    ...
```

必须用 `math.isqrt`，不能用 `int(n ** 0.5)`。 n 有 1024 位， 而 float 只有 53 位精度，`n ** 0.5` 会给出一个完全错误的近似值， 然后你拿它平方去比 n，**永远不相等。**

而且它**不报错** —— 只是安静地什么都找不到，你会以为语料里没有这一类。 这是「静默失败」，本实验里最贵的一类 bug。 检测它的唯一办法是：**先自己算一遍应该有几台。**

(4) 完全平方那条路的 φ，用的还是你在任务一写的 `totient`。

这里故意没有给你写好的公式：上面那条路（共享素因子）的 φ 是写好的 （那里 p ≠ q，没有歧义）；这一条路上要你自己选分支去调用 —— `p = q` 时会走进 `totient` 里的第二个分支。

如果你在任务一里把那一支写成了 `(p-1)²`，**现在就是它还债的时候** —— 而且这次不会有任务一那样的即时报警了，只有最后解不开的 PDF。 这正是任务一那个自检块存在的理由。

(5) 做完必须自查。

```
python verify.py              # 性质检查：字段、范围、往返、分类自洽
python verify.py --appendix-b # 再加一层：与附录 B 的承诺值逐条核对
```

`verify.py` **不告诉你答案**，只检查性质：d 是否落在 `[0, φ)`、 承诺值是否自洽、加解密是否往返成立、分类划分是否自洽。 它验 φ 时**不读你 JSON 里的 p、q**（本包的 JSON 契约里就没有 p、q）—— 它自己从 `data/moduli.txt` 重新算一遍因子，再拿去和你的 d 对。 **验证方不能向被验证方索取证据** —— 否则你 p、q 错了， 两边会一起错、互相印证通过，那这个检查就白做了。

想确认「一个不多、一个不少」，才加 `--appendix-b`。

> **建议先把分析做完再跑 `--appendix-b`。** 数不对本身是一条线索 —— 它会告诉你漏掉了哪一类。

5.4 任务四　解开那份通知【4 处填空】

目的

用你恢复出来的私钥，把 `data/homework.pdf.enc.asc` 解开， 拿到那份《密钥轮换通知》。

收件方就是那 **6 台**可分解设备中的一台 —— 但你不需要猜是哪一台： 逐个私钥试过去，只有正确的那把能让 PKCS#1 填充合法。 **用性质筛选，而不是用猜测筛选。**

命令

```
python task4_decrypt.py
python decrypt_hw.py --auto      # 兜底：拿 weak_devices.json 里的私钥逐个试
```

实测输出

```
================================================================
任务四：解开 PBP 文件
================================================================
RSA 密文        : 1023 位
IV              : d941f224df4ef958fcd07f26a82f93b0
AES 密文体      : 16,912 字节

手上有 6 把私钥，逐个试过去：

  设备 [···] : 不是这把
  设备 [···] : 不是这把
  设备 [···] : 不是这把
  设备 [···] : ✓ 成功

----------------------------------------------------------------
使用设备 #[···] 的私钥 —— 这把私钥是你自己算出来的。
----------------------------------------------------------------

已写出     : ...\data\homework.pdf
字节数     : 16,896
SHA-256    : d3c1c5b3b6bc7577ff19f6aa1a21e37ad473e9d1b6b5997450e80f2da3dc13d5

打开它     : start homework.pdf          （Windows）
             open  homework.pdf          （macOS）

到附录 B 核对这个 SHA-256。
```

要求

空	在哪	写什么
FILL[1]	decrypt_with	私钥运算 pow(c, d, n)（一行）
FILL[2]	decrypt_with	把整数还原成**定长**字节串 —— 这就是 EM 块
FILL[3]	decrypt_with（try 内）	剥掉 PKCS#1 v1.5 填充，取出会话密钥
FILL[4]	decrypt_with	**反向确认**：不成立的那个条件

关键结论

(1) 反向确认要加密的是整块 em，不是 session_key。

```
EM = 0x00 || 0x02 || PS || 0x00 || session_key
└──────────────────── RSA 作用在这一整块上 ────────────────────┘
```

```
pow(os2ip(em), e, n) == c            # ✓ 正确
pow(os2ip(session_key), e, n) == c   # ✗ 永远是 False
```

后者对一把**完全正确**的私钥也会返回 `False`。你会以为 d 错了， 然后去改一个本来没错的地方 —— 这是本任务最浪费时间的坑。 报告 §五 有一问就落在这里。

(2) `FILL[2]` 必须是定长**的编码。**

`i2osp(em_int, byte_len(n))` 里的第二个参数「EM 该有多长」不是可选的： 图省事写成「最短能装下就行」，EM 开头的 `0x00` 会被吃掉 —— 而 PKCS#1 要求第 1 字节正是 `0x00`。后果是**每一把**钥匙都被判为 填充不合法。你会顺着错误信息去怀疑自己的 d 算错了，方向完全跑偏。

(3) PDF 阅读器能打开 ≠ 你做对了。

`aes_soft.cbc_decrypt` 只解密、**保留** PKCS#7 填充； `decrypt_hw.aes_cbc_decrypt` 解密并**去掉**填充。用错会多出 1~16 字节， **PDF 照样打得开**，但字节数和 SHA-256 对不上附录 B。

这是一个「看起来成功了」的失败 —— 最需要警惕的一类。 **核对 SHA-256，不要只看到文件能打开就交。**

(4) 你没有分解任何 1024 位的数。

回顾一下你做过的事：

- 一次 499,500 对的最大公约数扫描；
- 1000 次开方检查（找完全平方）；
- 然后把 6 台设备的私钥全部恢复了出来。

这就是共享素因子的全部威力 —— 攻击的不是 RSA 的数学，是它的**实现。**

打开那份 PDF。 它是一份真实的运维通知 —— 注意它怎么区分那几类发现，以及它对「发现者有什么义务」的说法。

---

6. 报告要求

6.1 提交清单

#	文件	说明
1	`data/hits.json`	原始文件
2	`data/weak_devices.json`	主交付物。6 把私钥 + 分类结果
3	`data/homework.pdf`	解开的那份通知
4	`code/` 里你修改好的4个文件	完整代码随报告一起交
5	实验报告（Word）	按模板填写，见 §6.2

6.2 实验报告

参考实验报告模板完成实验报告内容。

附录 A　排错：

环境和运行

现象	原因	怎么办
`ModuleNotFoundError: No module named 'fill'`	用了 `python -I` 或 `-E`	去掉那两个参数。`-I` 会把脚本目录移出 `sys.path`
打印出来是乱码	终端编码不是 UTF-8	用 `PYTHONIOENCODING=utf-8 python -X utf8 xxx.py`
`python` 是 Microsoft Store 的转发器	Windows 的 App Execution Alias	用 `py -3 xxx.py`，或关掉那个别名
中文在报错里变成 `\xXX`	GBK 控制台	`fill.py` 已经处理了这个，正常不用管

附录 B　真值数据（对答案用）

B.1　承诺值

项	SHA-256 / 值
可分解设备集合	`09a8f58cf59d6085f84d5f82815d413a4946a6dcd4eb4705180d9444ba10a84b`
模数重复设备集合	`a6f9983507b74906a1a825a0907e81b91fd5e6630083ce1fb0802e7c18f4f207`
完全平方设备集合	`064c3e311ef63912b0cc91db9681ce2d301c3e76c447febf8faa303de38cc005`
私钥总账（设备号‖指纹）	`c09a70578f5e2abda10a3178714e9f6289a8a7c702718150c1291167a6a90b3c`

6 把私钥的承诺值指纹（不含设备号，按字典序排列）：

```
03371cecdb6387e1
1a6a777e738b0d5d
6b7ea890b40fb4b0
7c2b2f26bdfe447b
9b8bf2362f8a5442
a6283bd6d83be7a6
```

B.2　第四个数：异常合计

本实验的核心是「三个数不相等」：命中**条数**、命中涉及**台数**、真正**可分解台数**。而这第四个数**不由任何程序打印** —— 任务三只印前三个数，「异常合计」是你在报告 §四 4.1 里自己算出来的。 如果你还没动手算，先别往下看这张表。

项	值
可分解台数	6 台
模数重复台数	2 台
异常合计	**8 台**

「异常」是「可分解」与「重复但拆不开」两个集合的**并集**。

B.3 加密文件与语料的完整性

项	值
`homework.pdf` 明文 SHA-256	`d3c1c5b3b6bc7577ff19f6aa1a21e37ad473e9d1b6b5997450e80f2da3dc13d5`
`homework.pdf` 明文字节数	16,896
`homework.pdf.enc.asc` SHA-256	`5ea0eac8f519c3e90770366e5f0546879804314cbae1c8eb21761b97a5831f8e`
`moduli.txt` SHA-256	`a49e69c89f744c12a908efa3ed0c4ce34ae90fbeae072e55935208019ab6e192`

第一条是任务四的**硬性验收标准**：解出来的 PDF 必须**逐字节**等于它。 文件能打开不算过关。

附录 C 知识点映射

本实验里的东西	对应课程知识点	具体在哪
`egcd` / Bézout 恒等式	扩展欧几里得算法	任务一 `FILL[1]`~`FILL[4]`
`modinv` / 逆元存在条件	模运算、乘法逆元	任务一 `FILL[5]`、`FILL[6]`
负系数归约	模运算的规范形式	任务一 `FILL[7]`
$\varphi(n) = (p-1)(q-1)$	欧拉函数	任务一 `FILL[8]`、`FILL[9]`
$\varphi(p^2) = p(p-1)$	欧拉函数在素数幂上的取值	任务一 `FILL[10]`；任务三 `FILL[9]`
`gcd(n₁, n₂)` 拆 RSA	最大公约数、RSA 结构	任务二 `FILL[2]`；任务三 `FILL[6]`
e 与 φ(n) 互素	RSA 密钥生成的前提	任务一 `FILL[6]`
$d = e^{-1} \bmod \varphi(n)$	RSA 私钥	任务二 `FILL[6]`；任务三 `FILL[8]`
m^(ed) ≡ m (mod n)	欧拉定理	任务二的往返验证（报告 §三 有一问）
`isqrt` / 完全平方	整数开方、数论算法	任务三 `FILL[4]`、`FILL[5]`
PKCS#1 v1.5 填充	公钥密码标准	任务四 `FILL[2]`、`FILL[3]`
混合加密（RSA + AES）	对称/非对称结合	任务四全流程
C(n,2) 与 $O(n^2)$	算法复杂度分析	任务二的 499,500 对（背景）
实现常数 vs 算法复杂度	工程性能	任务二：`math.gcd` 与手写欧几里得的对比

附录 D 填空位清单（核对用）

共 29 处。 每填完一处打个勾。用 `python run_all.py --progress` 核对进度。

task1_math.py（10 处）

- [ ] FILL[1] egcd：(old_s, s) 的初值
- [ ] FILL[2] egcd：(old_t, t) 的初值
- [ ] FILL[3] egcd 循环体：同时更新 (old_s, s)
- [ ] FILL[4] egcd 循环体：同时更新 (old_t, t)
- [ ] FILL[5] modinv：一次取出 egcd 的三个返回值
- [ ] FILL[6] modinv：g != 1 时抛 ValueError
- [ ] FILL[7] modinv：把 x 归约到 [0, m) 再返回
- [ ] FILL[8] totient：分支条件
- [ ] FILL[9] totient：普通情形 p ≠ q
- [ ] FILL[10] totient：p == q（★ **$(p-1)^2$ 陷阱**）

task2_pair.py（6 处）

- [ ] FILL[1] scan_pairs：内层循环的起点
- [ ] FILL[2] scan_pairs：求两个模数的最大公约数
- [ ] FILL[3] scan_pairs：命中的判断条件（只写 g > 1）
- [ ] FILL[4] scan_pairs：把 (i, j, g) 记进 hits
- [ ] FILL[5] break_pair：把三种 g 的情形分开（**一整块代码**）
- [ ] FILL[6] break_pair：d = modinv(e, ϕ)

task3_scan.py（9 处）

- [ ] FILL[1] classify：一条命中属于哪一类（★ **价值最高**）
- [ ] FILL[2] classify：把设备 i、j 都记进 shared
- [ ] FILL[3] classify：把设备 i、j 都记进 duplicate
- [ ] FILL[4] find_perfect_squares：n 的整数平方根
- [ ] FILL[5] find_perfect_squares：判断 + 记录（**一小块**）
- [ ] FILL[6] recover_keys：与搭档求 gcd，得到 p
- [ ] FILL[7] recover_keys：q = n // p
- [ ] FILL[8] recover_keys：d = modinv(e, ϕ)
- [ ] FILL[9] recover_keys：phi = totient(p, q)（**p == q 的陷阱**）

task4_decrypt.py（4 处）

- [ ] FILL[1] decrypt_with：pow(c, d, n)
- [ ] FILL[2] decrypt_with：定长还原 EM 块
- [ ] FILL[3] decrypt_with：剥掉 PKCS#1 填充，取出会话密钥
- [ ] FILL[4] decrypt_with：反向确认（加密**整块** em）

附录 E　一页速查

命令

```
python check_env.py                  # 环境自检（课前作业，末尾附填空进度）
python task1_math.py --selftest      # 任务一
python task2_pair.py                 # 任务二
python task3_scan.py                 # 任务三【主交付物】
python task3_scan.py --adopt-hits    # hits.json 丢了？用它现场重扫
python verify.py                     # 自查（性质检查）
python verify.py --appendix-b        # 自查 + 与附录 B 核对
python task4_decrypt.py              # 任务四
python decrypt_hw.py --auto          # 任务四的黑盒兜底
python run_all.py                    # 一键跑全部
python run_all.py --progress         # 填空进度表
```

关键公式

$\gcd(n_1, n_2) = p$	共享素因子 —— 直接给出 p
$\phi(pq) = (p-1)(q-1)$	一般情形（$p \neq q$）
$\phi(p^2) = p(p-1)$	★ 不是 $(p-1)^2$
$d = e^{-1} \bmod \phi(n)$	私钥
$m^{ed} \equiv m \pmod n$	往返验证的依据
$C(n, 2) = n(n-1)/2$	两两对数

三个「别混」

别混	为什么
命中**条数** vs 命中**台数**	3 台互相共享时产生 3 条命中
命中台数 vs **可分解**台数	完全平方一条命中都没有；模数重复有命中但拆不开
异常 vs **可恢复私钥**	报告里必须分栏写，否则运维会轮换错批次

最容易踩的坑

坑	症状	唯一防线
元组赋值拆成两句	不报错，答案全错	任务一 Bézout 自检
`int(n ** 0.5)` 代替 `isqrt`	不报错，一台都找不到	**先自己算应该有几台**
$\phi(p^2)$ 写成 $(p-1)^2$	不报错，60 分钟后解不开 PDF	任务一自检【五】（范围检查抓不住它）
命中判断里滤掉 `g == n`	不报错，重复这一类整个消失	任务二的填空注释；`--appendix-b` 会对不上